

JavaScript コーディング規約

作成者：小島佑太

目次

1. 概要	2
2. 適用	2
3. コーディング規約の設定基準	2
4. JavaScriptコーディング規約	3
4. 1 全般	3
4. 2 JavaScriptのシンタックス	3
4. 2. 1 演算子	3
4. 2. 2 関数定義	7
4. 3 コードレイアウト	9
4. 4 表題コメントの書き方	11
4. 5 データの定義	16
4. 6 命名	16
4. 7 実行文の書き方	16
4. 8 コメントの書き方	20
4. 9 移植性の高いプログラムの書き方	23
4. 10 ソースコードの修正	23
4. 11 ファイルの構成	23

1. 概要

本書は、学内フリーマーケットシステム・芝浦市場（以下、本システム）で製作するC言語のプログラムのコーディングに対して規則を定める。この規則にそってプログラムをコーディングすることにより保守性の向上や統一した品質を保持することを目的とする。

2. 適用

本システムで製作する JavaScript 言語プログラムに適用する。

3. コーディング規約の設定基準

- (1) JavaScript 言語で生じやすいコーディングエラーを減らすために利用可能なコーディングスタイルを限定する。
- (2) レビュー及び保守を行う上で、次の基準を満たすこと
 - 十分に読みやすいこと
 - 技巧に走った難解なプログラミングでないこと
 - 必要なコメント情報が記述されていること
- (3) レビュー時に、チェックできる記述であること

4. JavaScript コーディング規約

本章では、JavaScript のプログラムを作成する上で守るべき規約について説明する。規約については下線を引いている。

4. 1 全般

- (1) 1 モジュールを 1 ファイルとする。
- (2) 原則として、一行一文とする。
- (3) 一行は原則 1 2 0 文字以内とする。

4. 2 JavaScript のシンタックス

4. 2. 1 演算子

- (1) 多様な演算子と優先順位に気を付けること(表 1)。

優先順位	演算子の種類	結合性	演算子
19	グループ化	なし	<code>(...)</code>
18	メンバーへのアクセス	左から右	<code>...</code>
	計算値によるメンバーへのアクセス	左から右	<code>... [...]</code>
	new (引数リスト付き)	なし	<code>new ... (...)</code>
	関数呼び出し	左から右	<code>... (...)</code>
	オプショナルチェーン	左から右	<code>?.</code>
17	new (引数リストなし)	右から左	<code>new ...</code>
16	後置インクリメント	なし	<code>... ++</code>
	後置デクリメント		<code>... --</code>
15	論理 NOT (!)	右から左	<code>! ...</code>
	ビット単位 NOT (~)		<code>~ ...</code>
	単項 +		<code>+ ...</code>
	単項 -		<code>- ...</code>
	前置インクリメント		<code>++ ...</code>
	前置デクリメント		<code>-- ...</code>
	typeof		<code>typeof ...</code>
	void		<code>void ...</code>
	delete		<code>delete ...</code>
	await		<code>await ...</code>
14	べき乗(**)	右から左	<code>... ** ...</code>
13	乗算(*)	左から右	<code>... * ...</code>
	除算(/)		<code>... / ...</code>
	剰余(%)		<code>... % ...</code>

12	加算 (+)	左から右	<code>... + ...</code>
	減算 (-)		<code>... - ...</code>
11	左ビットシフト (<<)	左から右	<code>... << ...</code>
	右ビットシフト		<code>... >> ...</code>
	符号なし右ビットシフト (>>>)		<code>... >>> ...</code>
10	小なり (<)	左から右	<code>... < ...</code>
	小なりイコール (<=)		<code>... <= ...</code>
	大なり (>)		<code>... > ...</code>
	大なりイコール (>=)		<code>... >= ...</code>
	<code>in</code>		<code>... in ...</code>
	instanceof		<code>... instanceof ...</code>
9	等価 (==)	左から右	<code>... == ...</code>
	不等価 (!=)		<code>... != ...</code>
	厳密等価 (===)		<code>... === ...</code>
	厳密不等価 (!==)		<code>... !== ...</code>
8	ビット単位 AND (&)	左から右	<code>... & ...</code>
7	ビット単位 XOR (^)	左から右	<code>... ^ ...</code>
6	ビット単位 OR ()	左から右	<code>... ...</code>

5	論理 AND (&&)	左から右	<code>... && ...</code>
4	論理 OR ()	左から右	<code>... ...</code>
	Null 合体 (??)	左から右	<code>... ?? ...</code>
3	条件 (三項) 演算子	右から左	<code>... ? ... : ...</code>
2	代入	右から左	<code>... = ...</code>
			<code>... += ...</code>
			<code>... -= ...</code>
			<code>... *= ...</code>
			<code>... *= ...</code>
			<code>... /= ...</code>
			<code>... %= ...</code>
			<code>... <= ...</code>
			<code>... >= ...</code>
			<code>... >>= ...</code>
			<code>... &= ...</code>
			<code>... ^= ...</code>
			<code>... = ...</code>
			<code>... &&= ...</code>
			<code>... = ...</code>
			<code>... ??= ...</code>
	<code>yield</code>	右から左	<code>yield ...</code>
	<code>yield*</code>		<code>yield* ...</code>
1	カンマ/シーケンス	左から右	<code>... , ...</code>

表 4-1 演算子と優先順位

出展

https://developer.mozilla.org/ja/docs/Web/JavaScript/Reference/Operators/Operator_precedence

(2) 優先順位が分かりにくい場合は括弧でくくること。

<例>

```
if ( ( (day == D_SUN ) && ( !busy ) )  ||  nearConcert ) {  
    playGuitar();  
}
```

(3) 式が2行以上にまたがる場合、優先順位のもっとも低い二項演算子の直前で改行すること。

<例>

```
if ( barOpen  
    && ( haveMoney( myPocket ) || have_card( myPocket ))  
    && notMyCar ) {  
    goToBar();  
}
```

(4) 関数呼び出しが二行以上にまたがる場合は、カンマ“, ”の直後で改行すること。

<例>

```
mail = makeMail( message,  
                 his.address,  
                 his.name,  
                 my.address,  
                 my.name)
```

4. 2. 2 関数定義

次の例は、一般的な関数定義である。

<例>

<形式>

<具体例>

<pre>function func(p1, p2, …) { . . . }</pre>	<pre>function ticket(data, concert) { . . . }</pre>
---	---

- (1) function に続けて関数名を記述すること。
- (2) 関数名に続けて” (”、仮引数の並び、”)” を記述すること。
- (3) 関数定義の前に関数の説明、仮引数の型と説明、返り値の型と説明を書くこと。JSdoc の機能で、関数を呼び出すときに確認することができる。必要に応じて、@example タグや@throws, @deprecated タグ等を使用すること。

<例>

```
/**
 * @function func
 * @description この関数は***をする関数.
 * @param {型名} p1 - 説明
 * @param {型名} p2 - 説明
 * @returns {型名} 説明
 *
 * @since 2024-06-04
 * @author 名無ごんべ
 *
 * @version 1.2
 * @modified 2024-06-05 by 芝浦太郎 - 変更内容
 * @modified 2024-06-06 by 名無ごんべ - 変更内容
 */
function func( p1, p2, ... ){
}
```

- (5) 関数に対応する”{”は()の後におき、” }” は行の1カラム目におき、その後には何も書かないこと
 - (6) 関数の中の処理は簡潔にすること。100ステップ以内で収まることが望ましい。
 - (7) 各処理で共通に使われる関数は引数渡しによりデータを受け渡しすること。
 - (8) 関数名や変数名など識別子は英字で始まる英数字列にする。また、31文字以内とする。また、命名規則はキャメルケースに従うこと。
- 例：setNum()

4. 3 コードレイアウト

ソースファイルは通常下記の定義部を含む。

- ・インポート
- ・グローバル変数の定義
- ・ローカル変数の定義
- ・関数定義

これらはソースファイル上では次のような構成にすること。

<例>

一般のファイル	
モジュールの表題コメント	
インポート	
外部変数の定義	
ローカル変数の定義	
関数コメント	
function 関数名、仮引き数	
{	
内部変数の定義	
実行文	
}	

* 各々の関数定義の直前にその関数についての” 関数表題コメント” をつけること。

(1) 変数は、下記の二種類に分類できる。明確に区別して使用すること。

- ・他のファイルから見える必要のあるもの（外部変数）
 - ・外部変数はさらに二種類に分類でき、グローバル変数と共有 Enum 用オブジェクトに分かれる。
 - ・グローバル変数名の先頭には必ず機能 ID をつけること。機能 ID に関しては、”**内部仕様書 外部変数**” を参照のこと。できる限り使用しないこと。効果的に使用できる場合、内部設計書の外部変数一覧に追加し、全体に通知すること。
 - ・共有 Enum 用オブジェクト名の先頭には必ず EnumID をつけること。EnumID に関しては” **内部仕様書 外部変数**” を参照のこと。
 - ・外部からインポートできるように、宣言時、export をつけること。
- ・そのファイル内だけで見ればよいもの（ローカル変数）
 - ・export をつけなければ自動的にそのモジュール（ファイル）内からしかアクセ

スができない。

4. 4 表題コメントの書き方

モジュール, vue コンポーネント, クラス, 関数, メソッド, 定数に対して JSDoc に従って記述する. これにより, `vscode` の機能で関数等を呼び出すときに説明が表示される. これを書く目的は3つある. 1つ目は, **JavaScript** は動的型付け言語であるが, 内部の型を明示することでエラーを抑えること. 2つ目は, 変更を追いやすくすること. 3つ目は, 製作者以外がコードを見ると, わかりやすくすること.

- (1) 表題コメントは, 対象要素を記述する前に設けること. 具体的に, モジュールならばファイルの冒頭, 関数ならば関数定義の前に書く.
- (2) 内容は以下の通りとする.

a. モジュール

```
/**
 * @module モジュール名
 * @description 説明
 *
 * @since 開発日
 * @author 開発者名
 *
 * @version バージョン
 * @modified 変更日時, 名前, バージョン, 変更内容
 */
```

b. vue コンポーネント

```
/**
 * @component コンポーネント
 * @description 説明
 *
 * @since 開発日
 * @author 開発者名
 *
 * @modified 変更日時, 名前, 変更内容
 */
```

c. クラス

```
/**
 * @class クラス名
 * @description 説明
 *
 * @since 開発日
 * @author 開発者名
 *
 * @modified 変更日時, 名前, 変更内容
 */
```

d. 関数

```
/**
 * @function 関数名
 * @description 説明
 * @param 引数の説明
 * @return 返り値の説明
 *
 * @since 開発日
 * @author 開発者名
 *
 * @modified 変更日時, 名前, 変更内容
 */
```

e. メソッド

```
/**
 * @function 関数名
 * @description 説明
 * @param 引数の説明
 * @return 返り値の説明
 *
 * @since 開発日
 * @author 開発者名
 *
 * @modified 変更日時, 名前, 変更内容
 */
```

f. 定数

```
/**  
 * @const 定数のデータ型, 定数名  
 * @description 説明  
 */
```

各項目の書き方

a. 要素名

タグは, なんの要素に対して表題コメントを書くかによって変わる(下表).
また, モジュール名は, ファイル名から拡張子以下を除いたものとする.
例: @module deleteStudent

モジュール	@module
関数	@function
メソッド	@method
Vue コンポーネント	@component
クラス	@class
汎用的	@name

b. 説明

このプログラムは何を行うのかを端的に記述すること。書きすぎてはいけない。ファイルを部分的に修正しただけでコメントを修正する必要があるならば, そのコメントは詳しすぎる。

例: @description このモジュールは, サインアップした生徒のデータを DB に格納する機能を持つ。

c. 引数の説明

下のようなフォーマットに従って引数を説明する。特に説明は, 出力をするための引数なら, 明記すること。

@param {データ型} 引数名 - 説明

また, 引数を複数個持つ関数またはメソッドの場合, すべての引数に対して記述する。

例: @param {number} arg - ****を判定するための値

d. 返り値の説明

下のようなフォーマットに従って引数を説明する.

e. 開発日

YYYY-MM-DD 形式で記述する. 最初に作り始めた日時を記載し, それ以降は変更しない.

例: @since 2024-06-06

f. 開発者名

日本語で名前を記述する.

例: @author 芝浦太郎

g. バージョン

形式は「vX.Y」とし, 基点は v1.0 からとする.

上位桁は改修単位, 下位桁は変更単位の修正になる. v1.13 のよう, 下位桁の 10 以上の形式も認める.

例: @version v1.2

h. 変更日時, 名前, バージョン, 変更内容

@modified YYYY-MM-DD by 名前 - vX.Y 変更内容

の形式で記述すること. ただし, モジュール以外はバージョンを書かなくてよい.

複数回変更をしたら, そのたびに新たな@modified を追加し, バージョンの個数文縦に羅列すること.

例: @modified 2024-06-07 by 芝浦太郎 - 同じ生徒が繰り返しユーザー登録したときに, 登録済と出力するように変更.

i. 定数の説明

@const {データ型} 定数名

例として, 画像を掲載しておく. 上画像が JSDoc の記述で, 下画像が vsCode による JSDoc の表示である. ただし, JSDoc の記述が完全に決まる前に書いたもののため, 書き方が若干定義と異なる可能性があるが, 上記の書き方に従うこと.

```

77  /**
78   * @description サンプル関数
79   * @param {string} a - テスト文字列
80   * @returns {void}
81   * @example sample('test') => console.log('test')
82   *
83   * @since 2024-06-04(制作日)
84   * @author 開発者名
85   *
86   * @version v1.2
87   * @modified 2024-06-05 by 名前1 - 変更内容1
88   * @modified 2024-06-06 by 名前2 - 変更内容2
89   */
90  function sample(a){
91      console.log('sample');
92      console.log(a);
93  }

```

```

function sample(a: string): void

@description — サンプル関数
@param a — テスト文字列
@example
sample('test') => console.log('test')
@since — 2024-06-04(制作日)
@author — 開発者名
@version — v1.2
@modified — 2024-06-05 by 名前1 - 変更内容1
sample('test');

```

4. 5 データの定義

- (1) JavaScript は、動的型付け言語なため、データの型に注意し、一貫性を保つこと。
特に、関数間の受け渡し時は型を JSDoc に記述し、十分に注意すること。
- (2) 配列の範囲外アクセスを避けること。配列の長さを取得するメソッドを積極的に使う必要がある。
- (3) 変数の初期化は必ず行うこと。

4. 6 命名

- (1) グローバル変数名の先頭には必ず機能 ID をつけること。機能 ID に関しては、“内部仕様書 外部変数”を参照のこと。グローバル変数を追加するときは、内部設計書の外部変数一覧に追加し、全体に通知すること。
- (2) Enum 用オブジェクト
- (3) 関数名、メソッド名、変数名等はキャメルケースに従い、一文字目を小文字、それ以降は単語の頭のみ大文字にすること。例：sampleFunc(), sampleValue
- (4) ファイル名、クラス名は1文字目を大文字にし、それ以降はキャメルケースに従うこと。例：SampleClass, SampleFile

4. 7 実行文の書き方

- (1) 一行に二つ以上の文を書かないこと。
- (2) 条件文や繰り返し文の対象となる文は、単文でも“{”と“}”でくくること。
- (3) 余分な else は書かないこと。
- (4) switch 文の default 部は最後に記述すること。
- (5) switch 文で case 部に break 文がない場合には、break 文がないのが正しいとコメントすること。

<例>

```
switch ( day ){
    case D_WEEK:
        work();
        /* no break */
    case D_SUN:
        playTennis;
        break;
    default:
        nothing();
        break;
```



```
}
```

(6) 次のような場合には、空白を使用し、文を読みやすくすること。

- ・代入演算子の前後

<例>

```
sum += count(today);  
his->address = look_up( myFriend, his->name);
```

- ・具体的な効果が認められる場所

- ・論理演算子の前後

<例>

```
if (bar_open && ( have_money( my_name) || have_money( his_name))) {  
    go_to_bar();  
}
```

(7) var は自動生成されたコード以外で使用しない。

(8) const が適用できる場合、let より優先して使用する。

(7), (8)の詳しい説明は notion の JavaScript ポリシーに記載

(9) 積極的に enum を使用すること。しかし、JavaScript は enum をサポートしていないため、下の書き方でオブジェクトを定義して enum として使うこと（この書き方なら、要素をほかの値に変更できない）。加えて、enum 用オブジェクトの要素は全て大文字で記述し、単語間を”_”で区切って命名する。また、複数のモジュール間で使用するときは、export をつけて外部変数としての共有 Enum 用オブジェクトを定義する。その場合は、内部設計書 外部変数 に記載されているルールに基づき、EnumID を付与し、共有 Enum オブジェクト一覧に追加すること。

<例>

```
const Enum1_direction = Object.freeze({  
    UP: 0,  
    RIGHT: 1,  
    DOWN: 2,  
    LEFT: 3  
});
```

(10) その他の一般的な文の表現方法を以下に示す。
以下に示した表現方法に沿ってコーディングすること。

<選択文>

<pre>switch(s) { case c1: statement case c2: statement default: statement }</pre>	<pre>switch(coin) { case C_PENNY: value = 1; break; case C_NICKEL: value = 5; break; case C_DIME: value = 10; break; default: put("?"); value = 0; }</pre>
---	--

<繰り返し文>

<pre>while(c) { statement }</pre>	<pre>while(input()) { print(); append(); }</pre>
<pre>do { statement } while(c);</pre>	<pre>do { key = getKey(); found = search(key); } while(!found);</pre>
<pre>for(;;) { statement break; }</pre>	<pre>for(;;) { input(); classify(); if (is_command) break; append(); }</pre>

```
for(let ... in ...){
    statement
    .....
}
```

```
for(let value in array){
    sum += value;
}
```

```
...forEach((...) => {
    statement
    .....
});
```

```
array.forEach((element) => {
    sum += element;
});
```

<分岐点>

```
goto label;
.....
label:
.....
```

```
if(overflow) goto FATAL;
return(SUCCESS);
FATAL:
abort();
```

(注意) goto 文は原則として使用しないこと。

<単純文>

```
e;
```

```
total = sum( data, n);
```

<ブロック文>

```
{
    declarations

    statement
    .....
}
```

```
{
    let yy, mm, dd;

    toYyMmDd( data, yy, mm, dd);
    is_busy = schdule( yy, mm, dd);
}
```

(注意) 原則として、ブロック文はデバッグ用以外に使用しないこと。

<条件文>

```
if(e) {
    statement
    .....
}
```

```
if( !locked) {
    lock(myCabinet);
    locked = TRUE;
}
```

if(e) {	if(barOpen) {
statement	drink = "brandy";
.....	myCar = NG;
}	}
else{	else{
statement	drink = "coffee";
.....	myCar = OK;
}	}

if(e1) {	if(isDigit(c)) {
statements	digit++;
}	}
else if(e2) {	else if(isLower(c)) {
statements	letter++;
}	}
.....	else if(isUpper(c)) {
.....	capital++;
}	}
else{	else{
statements	other++;
}	}

4. 8 コメントの書き方

- (1) コメントはまとまった処理の主要箇所に挿入すること。
- (2) 変数名や型などのソースコードに強く依存した情報はコメントに含めないこと。
- (3) 関数定義の直前に関数の表題コメントを入れる。
- (4) 読みやすくするため、文のまとまった段落毎に空白行を入れること。
- (5) 追加忘れ等を防ぐために、適宜以下のタグを使用してコメントすること（該当しない場合はタグ無しでよい）。VSCode 拡張機能の BetterComments を導入すると、これらのタグがハイライトされる。

///**!** 注意すべきこと

///**?** 後から調査や確認が必要なところ

///**todo** 後から追加や変更をする必要があるところ

///***** 直接的な意味は無いが、他のコメントより少し目立たせたい場合

(6) 修正の履歴をソースコード内にコメントとしていれる場合は、以下のように記述をし、修正部分が重なった場合は過去のものを消去する。

- ・追加の場合。

```
// From. Added 修正者 yyyy.mm.dd
~~ 修正コード ~~
// To. Added 修正者 yyyy.mm.dd
を記述
```

- ・変更の場合。

コード中の該当部分を削除し、

```
// From. Changed 修正者 yyyy.mm.dd
~~ 修正コード ~~
// To. Changed 修正者 yyyy.mm.dd
を記述
```

- ・削除の場合。

コード中の該当部分を削除するのみ。

<例>

```
if (flag) {
    data = MSL;
}
else {
// From. Added 芝浦 太郎 1996.7.16
    set_data(data);
// To. Added 芝浦 太郎 1996.7.16
    data = FTR;
}

// From. Changed 芝浦 太郎 1996.7.16
/* 前回値をチェックする処理 */
if (pre_flag) {
    data = HELO;
}
```

```
else {  
    data = BMBR;  
}  
// To. Changed 芝浦 太郎 1996.7.16
```

(7) データ定義にはコメントとして、データの意味を記入すること。

<例>

```
int    error_code;          // エラーコード
```

(8) 理解を助けるため、適宜、日本語（漢字かな）を使用する。

4. 9 移植性の高いプログラムの書き方

移植性を高めるため、以下の点に注意すること。

- (1) ハードウェア, ブラウザ, 特定の動作環境に依存する機能を極力使用しないこと.
- (2) 構造化されたプログラム設計をすること. (モジュール化, 関数やクラスの適切な使用)

4. 10 ソースコードの修正

- (1) 修正の履歴をソースコード内にコメントとして入れること。また、表題コメントに改訂の履歴を入れること。
記述方法は、4.4 及び 4.8(6)に従うこと。構成管理についてのドキュメント参照のこと
- (2) バージョン管理は git hub で行う。commit 時のメッセージは必ず書くこと。

4. 11 ファイルの構成

- (1) 1 モジュール 1 ファイルとすること.
- (2) ファイル構成は以下のよう to すること.

```
shibaura-ichiba/  
└ client/ クライアント PC 上で動く vue プロジェクト  
  │  
  │ └ src/ ソースファイルや画像ファイル等  
  │   │  
  │   │ └ assets/ 画像ファイル, フォント等の静的なリソース  
  │   │  
  │   │ └ components/ サイズが小さく(ボタン等)再利用可能な Vue コンポーネント  
  │   │  
  │   └ ┬ views/ 各ページにあたる Vue コンポーネント C1 UI 処理部の中核  
  │     │  
  │     └ server/ サーバー上で動く node, express プロジェクト  
  │       │  
  │       │ └ components/ 各コンポーネントのディレクトリ  
  │       │  
  │       │ └ informationManagementLayer/ 情報管理層のコンポーネント  
  │       │   │  
  │       │   └ (各コンポーネントディレクトリ) コンポーネントに属すモジュール  
  │       │  
  │       │ └ processingLayer/ 処理層のコンポーネント  
  │       │   │  
  │       │   └ (各コンポーネントディレクトリ) コンポーネントに属すモジュール  
  │       │  
  │       └ views/ ejs を含み, 動的な HTML を生成する  
  │  
  └ ┬ routes/ URL に対するルートハンドラを定義
```